Arhitecturi orientate pe servicii

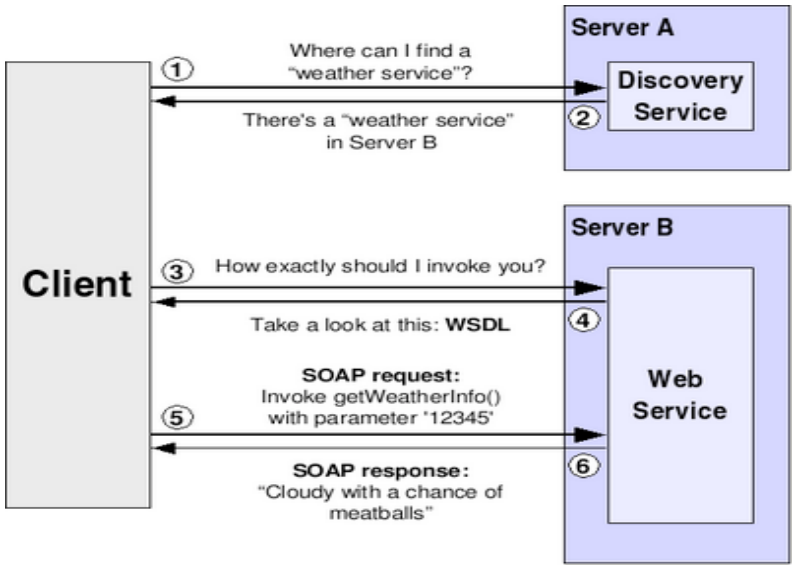
Web Services

Cuprins

- Servicii Web
- Reprezentarea si transportul mesajelor, SOAP
- Descrierea serviciilor, WSDL
- Serviciul director pentru servicii, UDDI
- Comparatie REST si SOA

Arhitecturi orientate pe servicii

Un serviciu software este definit ca o functie auto-continuta, care nu depinde de contextul sau starea altor servicii. ("A well-defined, self-contained function, which does not depend on the context or state of other services".)
Un sistem a carui arhitectura este bazata pe servicii software (SOA - Service Oriented Architecture) este o colectie de procese care se coordoneaza intre ele astfel incat un proces furnizor de servicii (server) poate fi folosit de un alt proces consumator de servicii (client). De regula, procesele nu se cunosc apriori intre ele explicit. De aceea, pentru a comunica procesele trebuie sa se regaseasca folosind mecanisme specializate. De exemplu, in paradigma publish / subscribe, furnizorul publica o descriere a serviciului pe care il ofera. In acelasi timp, consumatorul subscrie la anumite categorii de servicii de care are nevoie. Cand un astfel de serviciu este publicat de un furnizor, consumatorul este notificat. SOA nu este legata de o tehnologie specifica, putand fi implementata folosind modelele REST (Representational State Transfer), RPC, CORBA, DCOM sau Web Services. OASIS (Organization for the Advancement of Structured Information Standards) a elaborat modelul de referinta al SOA.



Elemente ale arhitecturii serviciilor Web

Asa cum am precizat anterior, publicarea serviciilor si notificarea clientilor permite regasirea si folosirea dinamica a serviciilor. O alternativa este folosirea unui serviciu special de descoperire a serviciilor. Serviciul director distribuit pentru servicii Web UDDI (Universal Description, Discovery, and Integration) ofera ambele mecanisme, atat pe cel de publicare/subscriere/notificare cat si pe cel de descoperire. In figura precedenta este ilustrat mecanismul de descoperire. Clientul se adreseaza serviciului de descoperire

cerand informatii despre un serviciu de *stare a vremii* si primeste ca raspuns informatii despre serverul care ofera un astfel de serviciu. Clientul interogheaza serverul cerand informatii depsre serviciu si primeste o *descriere a interfetei* serviciului intr-o forma care poate fi procesata de maşină, WSDL - Web Service Description Language. Clientul alcatuieste o invocare conforma cu specificatia WSDL si o transmite serverului folosind un protocol simplu, SOAP - Simple Object Access Protocol. Informatiile despre starea vremii sunt transmise de server ca raspuns la interogarea clientului.

Caracteristici luate de la servere Web

Serviciile Web imprumuta si extind elemente din abordari anterioare ale modelului client-server. De la serverele Web, preiau protocolul cerere-raspuns HTTP, care suporta comunicarea acestora cu clienti avand o functionalitate generala (browser) carora le furnizeaza rezultate intr-o forma usor de prelucrat pentru afisare pe ecranul utilizatorului (HTML). Este posibila descarcarea si excecutia unui program (applet) la client pentru functionalitati care depasesc simpla vizualizare a paginilor Web. O caracteristica importanta este abilitatea de a folosi cereri HTTP pentru punerea in executie a unui program la server, raspunsul transmis clientului fiind rezultatul executiei acestui program. Este important de subliniat diferenta intre *servere Web* si *servicii Web*: un server Web furnizeaza un serviciu HTTP elementar in timp ce un serviciu Web poate furniza un serviciu complex specificat de interfata sa.

Serverele Web au o modalitate universala de identificare, localizare si acces la resurse: URI (Uniform Resorce Identifier), URL (Uniform Resource Locator) si URN (Uniform Resource Name). URI este folosit pentru identificarea coerenta, procesabila de programe a resurselor Web sau a altor resurse Internet (de ex. cutii postale). Valoarea lui poate fi un URL sau un URN. URL contine informatii folosite in localizarea si accesul resurselor, iar URN sunt folosite ca nume independente de locatie pentru resurse (de exemplu, urn:ISBN:0-201-62433-8 identifica o carte in schema de nume standard ISBN). Modelul arhitectural REST, care sta la baza sistemului Web, este folosit in unele servicii Web (de exemplu, in cele furnizate de Amazon).

Serviciile Web adauga forme de cooperare mai bogate si structurate intre client si server. Astfel, un program al unui client dintr-o organizatie poate interactiona cu un server din alta organizatie fara supervizare umana. Se pot dezvolta, astfel, aplicatii complexe prin servicii care integreaza alte servicii. Datorita modului de interactiune, serviciile Web nu pot fi accesate direct de browsere. Reprezentarea externa a informatiilor si alcatuirea mesajelor (marshalling) schimbate intre servicii Web este facuta in XML. Prin acestea, serviciile Web suporta heterogeneitatea la nivelul limbajelor de programare, sistemelor de operare (diferite pentru client si server) si altele.

Caracteristici imprumutate de la RPC/RMI

De la apelurile de proceduri sau de metode la distanta (RPC, RMI …) sunt imprumutate doua elemente fundamentale. Primul se refera la descrierea interfetelor in limbaje declarative neutre cum sunt *Interface Definition Notation (IDN) folosit DCE RPC, Interface Definition Language (IDL) adoptat in CORBA* si altele. Avantajele majore sunt : separarea descrierii serviciului de implementarea sa, care permite integrarea cu diverse limbaje de programare; generarea automata a codului pentru alcatuirea mesajelor (pe baza cunoasterii formatului extern al datelor folosit in mesaje); si folosirea invocarilor dinamice pe baza unor descrieri generice a serviciilor.

Al doilea aspect major se refera la mecanisme pentru identificarea, localizarea, legarea (bind) si comunicarea cu serverul, fara de care combinarea dinamica a serviciilor nu ar fi posibila.

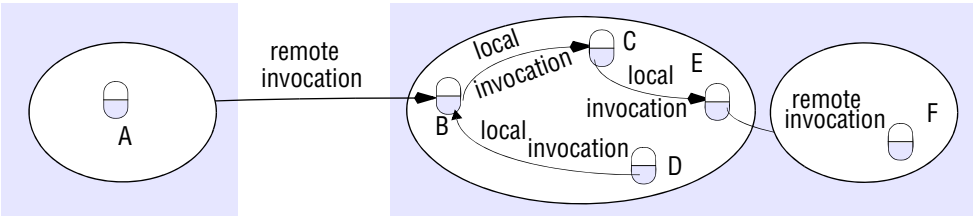
Modelul de obiecte distribuite

Concepte fundamentale

Un obiect poate invoca metode ale unui obiect la distanta daca cunoaste *referinta* sa *remote*. Un obiect are o referinta unica, indiferent daca este local sau *remote*. Un client poate obtine referinta unui obiect in mai multe moduri:

- La *crearea* obiectului – clientul poate crea un obiect şi obţine referinţa sa. Pentru a crea un obiect in CORBA, clientul invocă o operaţie a unui obiect **fabrică de obiecte**. Crearea întoarce o referinţă de obiect.
- Prin serviciul de *cataloage* (directory service) – permit obţinerea de referinţe, după nume, respectiv după proprietăţi.
- Prin conversia la/de la şiruri de caractere – o referinţă de obiect poate fi convertită în şir de caractere şi înapoi în referinţă putând fi utilizată după conversie, atât timp cât obiectul există.

Referintele obiectelor pot fi transmise ca argumente si rezultate ale invocarilor de metode. Astfel, la invocarea unei metode se transmit: referinta obiectului, numele metodei, argumentele



IDL - Interface Definition Language

Limbajul de definire a interfetelor, IDL permite specificarea semnaturilor unui set de metode (fara a specifica implementarea lor) si a tipurilor folosite in specificarea semnaturilor.

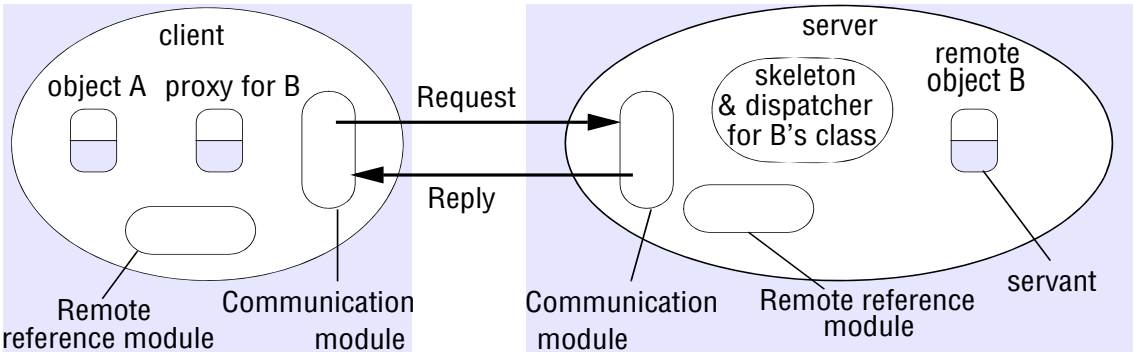
Exemplu CORBA

```
// In file Person.idl
        struct Person {
        string name;
        string place;
        long year;
};
interface PersonList {
        readonly attribute string listname;
        void addPerson(in Person p) ;
        void getPerson(in string name, out Person p); long number();
};
```

Pentru a face mai eficient transferul cererilor si raspunsurilor, parametrii sunt clasificati ca in (intrare), out (iesire) sau ambele. Astfel poate fi evitata transmiterea parametrilor out in comenzi si a parametrilor in in raspunsuri.

Implementarea RMI

Prezentarea descrie modul in care este asigurat apelul unei metode, facut de un obiect client (A in figura) si adresat unui *servant* reprezentat de un obiect *remote* (B in figura) care ruleaza intr-un proces server. Un servant este creat cand este instantiat obiectul remote si ramane in uz pana cand nu mai este nevoie de el, fiind sters sau eliminat prin garbage collection.



Operatiile sunt repartizate mai multor module. Modulele de comunicare executa protocolul de transmitere a cererii si raspunsului intre client si server, in conformitate cu o semantica stabilita, de exemplu *at-most once*. Modulele de comunicare folosesc trei campuri din mesajul primit: tipul mesajului, identificatorul cererii si referinta remote a obiectului. La server, el paseaza referinta remote modulului *Remote reference* si primeste referinta locala a obiectului invocat. In continuare, selecteaza dispecerul responsabil de clasa obiectului invocat, caruia ii paseaza referinta locala a acestuia.

Modulul *Remote reference* face traducerea intre referinte locale si *remote* folosind o tabela de corespondente intre ele.

Modulul *Proxy* asigura transparenta invocarilor de metode la distanta. El implementeaza metodele din interfata obiectului remote, dar in alt fel decat obiectul pe care-l reprezinta. Astfel el primeste invocari de la client, le converteste (marshalling) in mesaje pe care le transmite obiectului remote. De asemenea, primeste mesaje cu rezultatele invocarilor de la obiectul remote, le converteste (demarshalling) si le livreaza clientului care a facut invocarea.

Un server are un Dispecer si un Skeleton pentru fiecare clasa reprezentand un obiect remote. Dispecerul primeste mesaje de invocare de la modulul de comunicare, foloseste identificatorul operatiei pentru a selecta metoda potrivita din Skeleton si ii paseaza mesajul de cerere.

Skeletonul implementeaza metodele din interfata remote, dar implementarea este diferita de cea a metodelor din servant. Mai precis, el converteste (demarshalling) mesajul de invocare si invoca metoda corespunzatoare din servant. Apoi asteapta terminarea invocarii, converteste (marshalling) rezultatul (si exceptiile) in mesaje de raspuns pentru proxy

Clasele pentru proxy, dispecer si skeleton sunt generate automat de un compilator de interfete pe baza descrierii acestora. Invocarile de metode sunt dinamice, adica generate la executie. In acest scop se foloseste o forma generala de forma urmatoare:

invoke(obiect, metoda, in_param, out_param)

care include referinta la obiectul remote, numele metodei si parametrii furnizati de client.

Caracteristici servicii Web – comparatie cu modelul obiecte distribuite

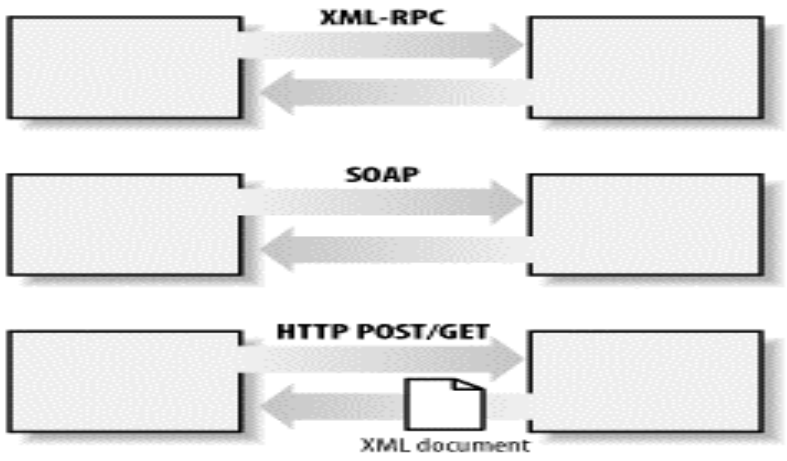
Exista multe similaritati intre implementarile modelului serviciilor bazat pe Web si cel cu obiecte distribuite:
- ambele fac separarea intre interfata unui serviciu si implementarea lui; o aplicatie client interactioneaza la distanta cu un serviciu WEB, care are o interfata specializata descrisa in WSDL folosind (XML).
- URI-ul unui serviciu Web este similar cu o referinta la un singur obiect *remote*
- ambele modele asigura transparenta invocarilor la distanta
- comunicarea intre clienti si servicii foloseste protocoale specializate si se bazeaza pe componente care asigura conversia invocarilor si a raspunsurilor in mesaje (marshalling / demarshalling)
- generarea codului pentru aceste componente se face automat, pornind de la specificarea interfetei serviciilor; astfel, in servicii Web, clientii / serverele pot citi si scrie mesajele direct in SOAP / XML dar detaliile pot fi ascunse de API-uri Java, Perl, Pyton, C++ care genereaza automat proceduri de marshalling / demarshalling
- regasirea si localizarea folosesc servicii de nume sau de atribute; serviciile Web pot fi gasite si localizate cu UDD.

Exista insa si diferente notabile. Astfel:
- obiectele folosesc un singur tipar de comunicare (intrebare/raspuns) intre client si server; serviciile Web folosesc SOAP, cu doua tipare, transmitere de mesaje asincrone si perechi de mesaje cerere-raspuns (similar RPC, RMI).
- un obiect poate creea dinamic obiecte remote si primi referintele lor pe care le poate folosi pentru invocarea metodelor la distanta; prin contrast, in serviciile Web, nu se pot creea instante la distanta, iar JAX-RPC (toolkit Java de servicii Web peste SOAP) nu permite pasarea referintelor de obiecte remote ca argumente sau rezultate; oricum, JAX-RPC mapeaza unele tipuri in Java pe definitii in XML folosite in SOAP si descrierile serviciilor
- la obiecte distribuite, serverul este o colectie de servants, obiecte care pot fi create dinamic la apelul unei metode; prin contrast, servicile Web nu suporta servants creati dinamic; ca urmare, implementarea interfetelor de servicii nu trebuie sa aiba constructori sau metode main.

Serviciile Web folosesc XML

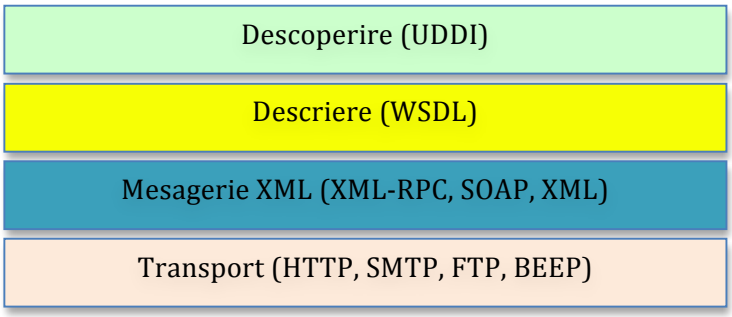
XML este folosit pentru comunicarea intre aplicatii, in mai multe variante (vezi figura urmatoare:

- XML-RPC
- SOAP
- HTTP GET/POST pentru transmitere de documente XML.



Arhitectura orientata pe servicii

Arhitectura serviciilor Web este ierarhica. Stiva de protocoale peste care sunt construite aplicatiile cuprinde nivelele ilustrate in figura urmatoare si descrise succint in continuare.
Nivelul transportului mesajelor intre aplicatii accepta diferite protocoale (HTTP, SMTP …). Dintre ele, BEEP (Blocks Extensible Exchange Protocol) ofera securitate, tratarea erorilor, dialogul initial intre entitati (handshake).
Nivelul urmator asigura codificarea mesajelor transmise intre client si serviciile Web. Ea foloseste reprezentarea textuala a XML, folosit in diverse tipare, care asigura accesul usor al operatorilor umani la descrierile mesjelor si, in consecinta, depanarea facila. Protocolul SOAP specifica regulile de utilizare a XML pentru impachetarea mesajelor si suportul tiparului cerere-raspuns in comunicarea intre client si servicii identificate prin URI-uri.
Nivelul descrierii interfetei publice a serviciilor foloseste standardul WSDL pentru a oferi informatii despre srviciile disponibile, modul de invocare si URL-urile serverelor. Evidenta serviciilor si mecanismul de publicare/subscriere si regasire sunt asigurate de UDDI.



SOAP – Simple Object Access Protocol

SOAP defineste o schema de utilizare a XML pentru reprezentarea mesajelor de cerere si raspuns sau pentru comunicarea de documente. El suporta schimbul datelor structurate in servicii Web in conformitate cu doua tipare. Primul este cel al mesajelor asincrone folosit pentru aplicatii in care procesarea dureaza mai mult si implica un schimb mai bogat de mesaje, inclusiv mesaje periodice de stare, detalii la finalul operatiilor etc. Un exemplu este procesarea unei cumparaturi.

Al doilea model, cerere-raspuns (RPC) asigura un dialog rapid in aplicatii online cum ar fi verificarea online a unui card bancar.

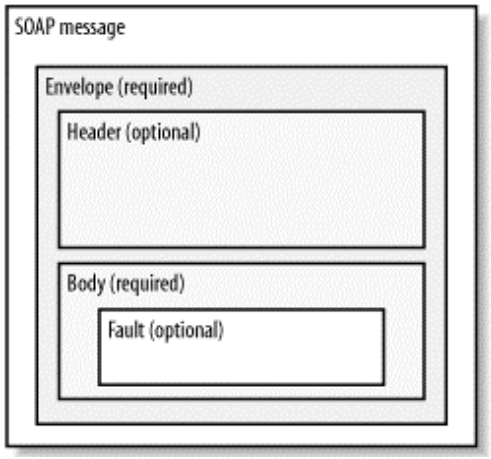
SOAP este o extensie a XML-RPC. El specifica cum se:

- foloseste XML pentru reprezentarea continutului mesajelor
- combina doua mesaje pentru a obtine tiparul cerere-raspuns
- interpreteaza continutul mesajelor
- cum se folosesc HTTP, SMTP … pentru comunicarea mesajelor

In continuare prezentam elementele principale ale unui mesaj SOAP. Precizam ca, desi mesajele se bazeaza pe scheme XML si, deci, au o reprezentare textuala usor de inteles de utilizatori, programatorii nu au de a face cu descrierea mesajelor datorita API-urilor SOAP implementate in diverse limbaje: Java, Perl, JavaScript, Pyton, C++, C#, Visual Basic.

Elementele principale ale unui mesaj XML SOAP

Um mesaj XML SOAP are doua elemente obligatorii: o *anvelopa* si *corpul* mesajului. O imagine a structurii mesajelor si elementelor optionale sunt prezentate in figura urmatoare. Antetul (Header - optional) este folosit in principal pentru a stabili contextul pentru un serviciu sau a pastra un log al operatiilor. Corpul (Body) mesajului poarta un document XML pentru un serviciu Web particular. O anvelopa poate include mai multe elemente Header si Body.



O **cerere** SOAP include metoda invocata si parametrii, asa cum se arata in exemplul urmator:

```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope
        xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:xsd="http://www.w3.org/2001/XMLSchema">
        <SOAP-ENV:Body>
                <ns1:getTemp
                xmlns:ns1="urn:xmethods-Temperature"
                SOAP-
                ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
                        <zipcode xsi:type="xsd:string">10016</zipcode>
                </ns1:getTemp>
        </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Cererea are un singur element obligatoriu Envelope, care include elementul obligatoriu Body.

Descrierea mesajelor foloseste XML namespaces pentru mai multe scopuri:
1. a diferentia versiunile SOAP prin specificarea in xmlns:SOAP-ENV (care contine definitiile SOAP pentru *Envelope, Header, Body*) a URI-ului corespunzator versiunii, dupa cum urmeaza:
- namespace URI pentru SOAP 1.1 este http://schemas.xmlsoap.org/soap/envelope/
- namespace URI pentru SOAP 1.2 este http://www.w3.org/2001/09/soap-envelope;

2. a specifica schema XML de codificare a datelor
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"

3. a dezambiguiza identificatorii din aplicatie specifici XMethods
xmlns:ns1="urn:xmethods-Temperature"

Cererea poate contine in Header si alte specificatii ca semnatura digitala pentru servicii protejate, un numar de cont pentru serviciile cu plata, etc.

Corpul

Elementul Body este folosit de regula pentru cerere si raspuns RPC, asa cum se arata in urmatorul exemplu. Pentru cerere, exemplul include un singur element Body care contine numele metodei apelate getTemp legat de namespace XMethods si URI-ul namespace-ul (fisierul care contine schema XML) relevant pentru descrierea serviciului. Acesta are ca sub-element parametrul zipcode cu tipul corespunzator schemei XML xsd:string si valoarea 10016.

Raspunsul SOAP

Un raspuns reusit are un element getTempResponse care returneaza o valoare de tipul xsd:float si valoarea (temperatura) in acest exemplu este 71.0 grade Fahrenheit. Numele procedurii are adaugat sufixul *Response*. Raspunsul utilizeaza aceleasi XML schema ca si cererea.

```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope
        xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:xsd="http://www.w3.org/2001/XMLSchema">
        <SOAP-ENV:Body>
                <ns1:getTempResponse
                xmlns:ns1="urn:xmethods-Temperature"
                SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
                        <return xsi:type="xsd:float">71.0</return>
                </ns1:getTempResponse>
        </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Raspuns la eroare

In cazul unei erori, raspunsul include un element Fault cu doua sub-elemente care specifica faultCode – clasa de erori Client insemnand ca cererea contine o eroare (de ex. nume metoda inexistenta) precum si un sub-element faultString care descrie eroarea in limbaj natural.

```
<?xml version='1.0' encoding='UTF-8'?>
```

```
<SOAP-ENV:Envelope
        xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
        xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
        xmlns:xsd="http://www.w3.org/1999/XMLSchema">
        <SOAP-ENV:Body>
                <SOAP-ENV:Fault>
                <faultcode xsi:type="xsd:string">
                SOAP-ENV:Client</faultcode>
                <faultstring xsi:type="xsd:string">
                        Failed to locate method (ValidateCreditCard) in class
                        (examplesCreditCard) at /usr/local/ActivePerl-5.6/lib/
                        site_perl/5.6.0/SOAP/Lite.pm line 1555.
                </faultstring>
                </SOAP-ENV:Fault>
        </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Tipuri de date

SOAP include un set de reguli pentru codificarea datelor. Uzual, regulile sunt implementate direct de toolkit-uri SOAP. SOAP foloseste

- specificatiile XML Schema elaborate de W3C
- propriile conventii (de ex. pentru tablouri si referinte).

Stilul de codificare este setat prin atributul SOAP-ENV:encodingStyle

SOAP 1.1 - http://schemas.xmlsoap.org/soap/encoding/
SOAP 1.2 - ttp://www.w3.org/2001/09/soap-encoding

Tipurile scalare

- includ string, float, double, integer
- tipuri definite de XML Schema.

In exemplul urmator, atributul xsi:type este setat la xsd:double:

```
 SOAP-ENV:encodingStyle="http://www.w3.org/2001/09/soap-encoding">
 <return xsi:type="xsd:double">54.99</return>
```

Tipuri compuse

Pentru tablouri, tipul xsi:type este un Array. Tabloul trebuie sa contina un atribut arrayType, necesar pentru a specifica tipul elementelor si dimensiunile tabloului. De exemplu, un tablou de 10 elemente double este specificat prin atributul:

arrayType="xsd:double[10]"

Pentru a specifica un tablou bi-dimensional de string-uri se foloseste agtributul

arrayType="xsd:string[5,5]"

Urmatorul exemplu este un fragment al unui raspuns SOAP, care descrie un tablou cu doua elemente de valori double. Fiecare element al tabloului este specificat ca un element item:

```
SOAP-ENV:encodingStyle="http://www.w3.org/2001/09/soap-encoding">
<return
        xmlns:ns2="http://www.w3.org/2001/09/soap-encoding"
        xsi:type="ns2:Array" ns2:arrayType="xsd:double[2]">
        <item xsi:type="xsd:double">54.99</item>
        <item xsi:type="xsd:double">19.99</item>
```

```
</return>
```

In cazul structurilor, pentru fiecare element este specificat un nume de acces unic si un tip:

```
<return xmlns:ns2="urn:examples" xsi:type="ns2:product">
        <name xsi:type="xsd:string">Red Hat Linux</name>
        <price xsi:type="xsd:double">54.99</price>
        <description xsi:type="xsd:string">
                Red Hat Linux Operating System
        </description>
        <SKU xsi:type="xsd:string">A358185</SKU>
</return>
```

In structura din exemplu se folosesc patru elemente de acces: name , price , description si SKU (Stock-Keeping Unit). Elementul name are tipul string.

SOAP peste HTTP

SOAP nu este legat de un protocol de transport specific, dar HTTP este cel mai popular. Cererile SOAP sunt transmise prin cereri HTTP, iar raspunsurile sunt primite prin raspunsuri HTTP. Desi pentru cereri se poate folosi HTTP GET, este preferat HTTP POST deoarece nu este limitat la numar de caractere (ca PUT).
Content-type pentru cereri si raspunsuri este text/xml. In plus, clientii trebuie sa specifice un antet (header) special SOAPAction. Acesta permite determinarea rapida a intentiei cererii fara a fi necesara examinarea corpului mesajului. Desi folsirea SOAPAction este obligatorie, valoarea asociata este depenenta de implementarea serverului. De exemplu,

```
SOAPAction: "urn:xmethodsBabelFish#BabelFish"
```

permite accesul la serviciul Translation de la AltaVista BabelFish, gazduit de XMethods
Chiar daca implementarea serverului nu cere un antet SOAPAction, clientul trebuie sa includa un string gol sau o valoare null:

```
SOAPAction: ""
SOAPAction:
```

Exemplu cerere

```
POST /perl/soaplite.cgi HTTP/1.0
Host: services.xmethods.com
Content-Type: text/xml; charset=utf-8
Content-Length: 538
SOAPAction: "urn:xmethodsBabelFish#BabelFish"

<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope
        xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
        xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
        xmlns:xsd="http://www.w3.org/1999/XMLSchema">
        <SOAP-ENV:Body>
                <ns1:BabelFish
                xmlns:ns1="urn:xmethodsBabelFish"
                SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
                        <translationmode xsi:type="xsd:string">en_fr</translationmode>
                        <sourcedata xsi:type="xsd:string">Hello, world!</sourcedata>
                </ns1:BabelFish>
        </SOAP-ENV:Body>
```

```
</SOAP-ENV:Envelope>
```

Exemplu Raspuns

```
HTTP/1.1 200 OK
Date: Sat, 09 Jun 2001 15:01:55 GMT
Server: Apache/1.3.14 (Unix) tomcat/1.0 PHP/4.0.1pl2 SOAPServer:
SOAP::Lite/Perl/0.50
Cache-Control: s-maxage=60, proxy-revalidate Content-Length: 539
Content-Type: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope
        xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/"
        SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
        xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
        xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
        xmlns:xsd="http://www.w3.org/1999/XMLSchema">
                <SOAP-ENV:Body>
                <namesp1:BabelFishResponse xmlns:namesp1="urn:xmethodsBabelFish">
                <return xsi:type="xsd:string">Bonjour, monde!</return>
                </namesp1:BabelFishResponse>
        </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Definirea interfetelor Serviciilor Web

Definitia interfetei este necesara clientilor pentru a putea comunica cu serviciile. O interfata de serviciu Web consta intr-o colectie de operatii ce pot fi executate de programe, obiecte, baze de date etc. In plus, furnizarea unei interfete permite combinarea operatiilor mai multor servicii pentru a realiza prelucrari complexe.

Serviciile Web sunt slab cuplate, intelegand prin aceasta ca exista dependente minime intre servicii. Proprietate sustinuta de programarea cu interfete, care separa interfata de implementare si sustine heterogeneitatea sistemului si orientarea pe interfete simple, generice. Serviciile Web au mai multe paradigme de comunicare oferind grade diferite de cuplare: mai tare la modelul request-reply, mai slaba la mesaje asincrone.

Elemente WSDL

Definitiile unei interfete sunt specificate ca parte a unei descrieri mai generale, exhaustive, care include toate elementele relevante pentru client. In fapt, descrierea este fundamentul unei intelegeri intre client si server privind serviciile oferite. Descrierile sunt facute in WSDL – Web Service Description Language si sunt folosite, in general, pentru a genera codul proxy la client, asigurand automat comportarea corecta a clientului in utilizarea serviciilor Web. WSDL permite aplicatiilor sa integreze cu usurinta noi servicii.

Descrierea serviciului in WSDL este mai flexibila decat in alte IDL. Metoda de comunicare este este lasata la decizia furnizorului de servicii, putand fi descrisa in termenii unor mesaje transmise (intr-un singur sens) intre client si server sau a unor combinatii de mesaje cerere-raspuns transmise la executia unei operatii suportate de server.

WSDL nu impune o implementare specifica a serviciilor, desi solutia frecventa este cea a serviciilor SOAP peste HTTP. De asemenea, specificarea URI in descrierea serviciului elimina nevoia unui “binder”, folosit in alte solutii middleware.

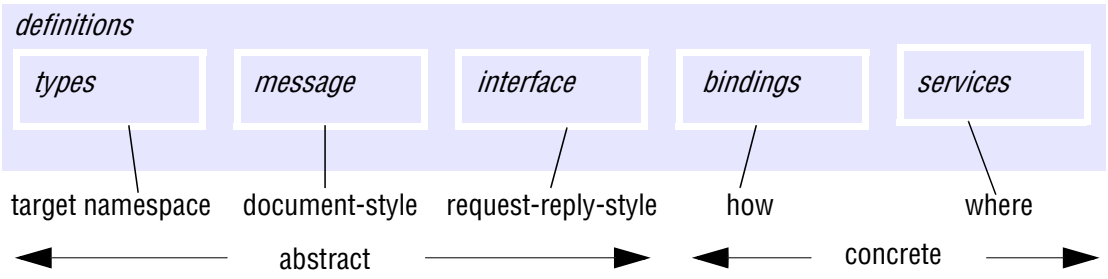
Descrierea are un element radacina (definitions) si elemente incluse impartite in doua categorii. Partea abstracta a descrierii include un set de definitii de tipuri folosite de serviciu, inclusiv tipurile valorilor communicate in mesaje. Setul de nume definite in sectiunea de tipuri a definitiei WSDL constituie spatiul de nume *target namespace*.

Sectiunea *message* descrie mesajele schimbate specificand numele mesajelor si, pentru fiecare mesaj, partile care se refera la parametri sau la valorile returnate.

Partea de *interface* descrie operatiile serviciului Web si, pentru fiecare operatie, specifica tiparul de mesaje schimbate (cerere, raspuns sau amandoua) si descrie mesajul de cerere si raspunsul corespunzator.

Partea *concreta* a descrierii specifica modul in care mesajele sunt transportate intre clent si server (*bindings* - de regula folosind servicii SOAP peste HTTP) si URI-ul folosit la invocarea serviciului (*services*).

The main elements in a WSDL description



Exemplu: serviciu Hello

Descriem aceste elemente folosind ca exemplu un serviciu “Hello” cu o singura functie disponibila public numita “Hello”.

```
<?xml version="1.0"?>
<wsdl:description
   xmlns:wsdl=http://www.w3.org/ns/wsdl
   xmlns:wsoap= "http://www.w3.org/ns/wsdl/soap"
   xmlns:hy="http://www.herongyang.com/Service/"
   targetNamespace="http://www.herongyang.com/Service/">
```

“description” este elementul radacina al documentului WSDL 2.0. El contine declaratii de namespaces folosite in document:

- xmlns:wsdl = namespace pentru toate numele de elemente folosite in limbajul de baza WSDL
- targetNamespace = furnizeaza un target namespace pentru toate numele definite in acest document
- xmlns:tns – furnizeaza un prefix pentru target namespace pentru toate numele definite in acest document (folosit pentru calificarea numelor prin prefix tns: pentru a se referi la nume din targetNamespace)
- xmlns:hy = folosit la tipuri
- xmlns:wsoap = furnizeaza wsoap ca prefix pentru numele de elemente din

extensia SOAP

Definirea tipurilor

```
<types>
     <xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
     targetNamespace="http://www.herongyang.com/Service/">
     <xsd:element name="Hello" type="xsd:string"/>
     <xsd:element name="HelloResponse" type="xsd:string"/>
     </xsd:schema>
 </types>
```

Defineste tipurile datelor folosite in mesajele de intrare/iesire ale serviciului. In exemplu, se foloseste XML Schema, suportata de WSDL 2.0. targetNamespace este necesar pentru a face disponibile tipurile de date si elementele de date din target namespace-ul documentului WSDL. In descriere nu sunt definite tipuri de date, dar sunt definite doua elemente de date, “Hello” si “HelloResponse”, ambele de tip “xs.string”.

- tipurile de elemente trebuie definite ca elemente singulare (nu ca secventa de elemente)
- tipul poate fi complex (elementul poate include alte elemente)
- xmlns:hy – prefixul hy: va permite referirea (la definirea interfetei) la Schema definita pentru tipurile mesajelor

Definirea interfetei

```
<wsdl:interface name="helloInterface" >
   <wsdl:operation name="Hello"
    pattern="http://www.w3.org/ns/wsdl/in-out"
    style="http://www.w3.org/ns/wsdl/style/iri">
    <wsdl:input messageLabel="In"
      element="hy:Hello" />
    <wsdl:output messageLabel="Out"
      element="hy:HelloResponse" />
   </wsdl:operation>
</wsdl:interface>
```

Defineste interfata ca o colectie de operatii a serviciului cu sub-elemente “operation”. O operatie este definita cu o secventa de mesaje “input” si “output” pentru a forma un tipar de cerere-raspuns. In exemplu, se defineste interfata “helloInterface” care are o operatie Hello cu pattern in-out (cand clientul trimite un mesaj in la server, acesta raspunde cu un mesaj out). Alte tipare predefinite sunt in-only, robust-in-only.

“input” este definit ca sa utilizeze elementul de date “Hello” iar “output” pentru a utiliza elementul de date “HelloResponse”. “style” ofera o lista de stiluri pentru operatie. In exemplu, se precizeaza ca mesajele pot fi serializate

Binding

```
<wsdl:binding  name="helloBinding"
 interface=”hy:helloInterface"
 type="http://www.w3.org/ns/wsdl/soap"
```

```
 wsoap:protocol="http://www.w3.org/2003/05/soap/bindings/HTTP/">
 <wsdl:operation ref="hy:Hello"
 wsoap:mep="http://www.w3.org/2003/05/soap/mep/soap-response"/>
</wsdl:binding>
```

"binding" specifica protocoolul de comunicatie. In exemplu, binding-ul are numele helloBinding si este legat de interfata helloInterface. El foloseste extensia WSDL SOAP pentru a specifica protocolul SOAP peste HTTP, care este mentionat in "protocol". Numele de binding definit aici este referit la definirea service endpoint.

"type" specifica formatul mesajului, aici SOAP 1.2. In fine, pentru operatia "Hello", wsoap:mep specifica tiparul schimbului de mesaje (MEP – Message Exchange Pattern), aici in-out.

Descrierea serviciului

```
<wsdl:service name="helloService"
  interface="hy:helloInterface">
  <wsdl:endpoint name="helloEndpoint"
  binding="hy:helloBinding"
  address="http://hello.example.com/2004/Service/HelloService"/>
</wsdl:service>

</wsdl:description>
```

"service" este folosit pentru a defini un serviciu cu unul sau mai multe sub-elemente "endpoint" pentru o interfata data. In exemplu, se defineste serviciul helloService, cu interfata helloInterface si cu un singur element endpoint, helloEndpoint. "endpoint"-ul este definit cu o adresa URL pentru binding-ul helloBinding. "service" este ultimul element din descrierea WSDL.

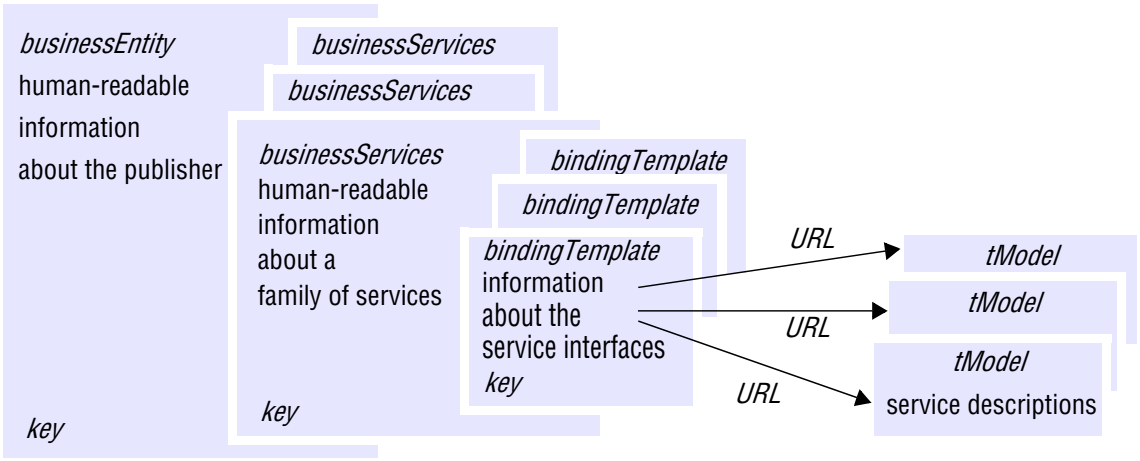
Utilizarea WSDL

Documentele WSDL pot fi accesate prin URI-uri de clienti si servere. Oricum, pentru a ocoli structurile complexe ale WSDL si a usura efortul utilizatorilor, generarea definitiilor WSDL se poate face prin instrumente cu interfata grafica. Astfel, Web Services Description Language for Java Toolkit (WSDL4J) permite crearea, reprezentarea si manipularea documentelor WSDL. Definitiile WSDL pot fi generate in interfete scrise in alte limbaje, de exemplu JAX-RPC.

Universal Description, Discovery and Integration service (UDDI)

Exista mai multe cai pe care clientii pot gasi descrierile de servicii Web. O solutie simla este pagina Web a unui furnizor de servicii. O solutie mai convenabila este folosirea unui serviciu director. UDDI ofera servicii de nume si director. Prima permite cautarea dupa numele serviciului (similar paginilor albe – traditionalele carti de telefon), cealalta permite cautarea dupa atribute, deci dupa categoria serviciului dorit (pagini aurii).

Datele sunt organizate ca patru structuri care permit incorporarea oricator informatii textuale, fiecare structura poate fi accesata individual printr-o cheie.



Organizarea datelor

Structurile au urmatoarele roluri:
businessEntity – descrie organizatii care furnizeaza serviciile (nume, adresa, activitati …)
businessServices – tine informatii despre un set de instante ale unui serviciu Web (nume, descriere scop de ex. agent turism)
bindingTemplate – tine adresa unei instante de serviciu Web si referinta la descrierea serviciului
tModel – tine descrierile serviciilor, de regula documente WSDL accesibile prin URI-uri

Cautarea

UDDI are o API pentru servicii de cautare, care ofera doua seturi de query, pentru
- gasirea unei entitati corespunzatoare unei chei; are forma:

get_xxx - *get_BusinessDetail, get_ServiceDetail, get_bindingDetail* si *get_tModelDetail*
- gasirea unor entitati corespunzatoare unui set de criterii, cu forma

find_xxx - find_business, find_service, find_binding and *find_tModel*

Rezultatul este un grup de nume, descrieri, chei si URL-uri.

Clientul care cunoaste o cheie particulara poate face o invocare get pentru a gasi direct entitatea respectiva. Altfel, clientul poate adresa invocari find succesive, mergand de la *find_business* la *find_binding*, urmand sa gaseasca o cheie care ii va permite sa regaseasca documentul WSDL potrivit.

UDDI ofera o interfata publish/subscribe/notify care furnizeaza abonatilor informatii despre entitatile de interes.

Publicarea

UDDI are o interfata pentru publicarea si actualizarea serviciilor Web. La prima publicare a unei structuri de date, i se asociaza o cheie in forma unui URI
- ex. uddi:abcd1.net:123

Serverul UDDI care publica serviciul devine proprietarul cheii. Serverul difuzeaza apoi informatia altor servere. Raspunsul la o cautare in UDDI este dat de un singur server, fara a interactiona cu alte servere din acelasi registru.

Registre

Un registru UDDI consta din unul sau mai multe servere, fiecare avand o copie a aceluiasi set de date. Modificarile unei structuri de date se fac la serverul proprietar, cel la care aceasta a fost publicata prima data. Proprietarul poate ceda acest drept unui alt server.

Schema de replicare este simpla:
- serverul care face modificarea notifica celelalte servere
- acestea cer modificarile pentru actualizare
- propagarea modificarilor foloseste vectori de timp

Folosirea este simpla:
- toate modificarile unei structuri de date se fac la acelasi server care propaga apoi actualizarile
- ca urmare, actualizarile sunt primite de un server in ordinea secventiala corecta
- nu se impune vreo ordine intre actualizarile facute de servere diferite.

Comparatie REST si SOA

REST (Representational State Transfer) este o arhitectura cu un set restrans de operatii de acces si modificare a unor resurse identificate prin URL-uri (atasate resurselor la crearea acestora): GET, PUT, DELETE and POST. Clientii le folosesc pentru a manipula resurse ale caror reprezentari sunt descrise in XML. Orientarea este pe manevrarea datelor si nu pe interfete, clientii primind intreaga stare a resursei si nu doar o parte a acesteia ca raspuns la o anumita operatie. Este interesnt de notat ca 80% din cererile adresate serviciilor Web de la Amazon sunt suportate de modelul REST si doar 20% folosesc SOAP.

Bibliografie

Acest document este bazat pe prelucrarea unor sectiuni din urmatoarele lucrari :

Ethan Cerami, Web Services Essentials. Distributed Applications with XML-RPC, SOAP, UDDI & WSDL. O'Reilly 2002
George Coulouris, Jean Dollimore,Tim Kindberg, Gordon Blair. Distributed Systems Concepts and Design, ed. 5, Addison Wesley